

Implementing LIME from Scratch with GPU Acceleration

Milestone 3

Parallel and Distributed Computing

Haris Khalid 28557

Aadesh Panjwani 29303

Hamna Hammad 29003

Bareerah Shahid 29385

Spring 2025

Contents

1	Project Overview	3
1.1	LIME Pipeline	3
2	Division of Tasks	3
3	Project Structure	4
3.1	Repository	4
3.2	GPU-Native Surrogate (m3_surrogate.py)	4
3.2.1	Algorithm: Weighted Least-Squares	4
3.2.2	Solver 1 — Closed-form (cuBLAS + cuSolver)	4
3.2.3	Solver 2 — Gradient Descent (custom CUDA kernels)	5
3.3	Explanation Extraction (m3_explanation.py)	6
4	Results and Validation	6
4.1	M1 Module Correctness (Re-confirmed)	6
4.2	Surrogate Training (Closed-form)	6
4.3	Surrogate Training (Gradient Descent)	6
4.4	Feature Attributions (Closed-form)	7
4.5	Validation Results	7
4.5.1	Feature Ablation	7
4.5.2	Multi-Seed Consistency Check	8
5	Performance	9
5.1	3-Way Pipeline Comparison ($N = 2048$, $F = 64$)	9
5.2	Benchmark Sweep	9
6	Design Decisions and Discussion	10
6.1	Linear Regression Instead of Neural Network	10
6.2	Closed-form vs. Gradient Descent	10
6.3	Ridge Regularisation	10
6.4	Minimal Data Transfer	10
6.5	GD Kernel Launch Overhead	10
7	Challenges and Limitations	11
7.1	GD Convergence on Ill-conditioned Problems	11
7.2	Binary Masking as the Perturbation Strategy	11
7.3	GPU Overhead at Small Scale	11
8	Conclusion	11

1 Project Overview

LIME (Local Interpretable Model-agnostic Explanations) is a post-hoc interpretability method that explains individual predictions of any black-box classifier by training a simpler, locally-faithful surrogate model.

Milestone 1 delivered GPU-accelerated perturbation generation, logistic regression inference (cuBLAS), and Euclidean/cosine kernel weight computation using custom CUDA kernels. Milestone 2 completed the pipeline by training a shallow PyTorch neural network surrogate and extracting gradient-based feature attributions. Milestone 3 replaces the PyTorch surrogate with a fully **GPU-native** implementation: no deep-learning framework is used at any stage.

Milestone	Surrogate	Frameworks	Status
M1	None (pipeline only)	Forbidden	Complete
M2	Shallow NN (PyTorch)	Permitted	Complete
M3	Weighted WLS (CUDA)	Forbidden	Complete

1.1 LIME Pipeline

For a given input instance $\mathbf{x}$, the full pipeline consists of five steps:

1. **Perturbation:** Generate N perturbed samples around $\mathbf{x}$ using binary feature masking.
2. **Inference:** Run the black-box model on all samples to obtain predictions $\hat{y}_i$.
3. **Weights:** Compute proximity weight $w_i = \exp(-d_i^2/2\sigma^2)$ for each sample.
4. **Surrogate training:** Fit a weighted surrogate on $(\tilde{\mathbf{x}}_i, \hat{y}_i, w_i)$.
5. **Explanation:** Extract surrogate coefficients as feature attributions.

Steps 1–3 were GPU-accelerated in M1. Step 4 used PyTorch in M2 and is fully GPU-native in M3. Step 5 now uses on-device GPU sorting.

2 Division of Tasks

- **Haris Khalid:** Designed the closed-form GPU solver (`solve_closed_form`) using CuPy cuBLAS matmuls and cuSolver LU decomposition. Integrated the end-to-end M3 pipeline in `m3_main.py` and produced all benchmark results.
- **Aadesh Panjwani:** Implemented all four custom CUDA kernels for the gradient descent solver (residuals, weighted gradient, parameter update, MSE loss). Added column normalisation to improve GD convergence.
- **Hamna Hammad:** Implemented GPU-side explanation extraction (`m3_explanation.py`): on-device top- k sort, ablation test, and coefficient/convergence visualisations.
- **Bareerah Shahid:** Designed and executed the consistency check across three perturbation seeds, produced the 3-way performance comparison, and documented design decisions.

All members collaborated on the numerical stability analysis and final report.

3 Project Structure

3.1 Repository

File	Responsibility
lime_common.py	Shared dataclasses, constants, utility functions
perturbation.py	CUDA kernels for mask generation and application (M1)
inference.py	GPU logistic regression forward pass (M1)
weights.py	CUDA distance and kernel weight kernels (M1)
m3_surrogate.py	CUDA kernels, closed-form and GD solvers, CPU reference
m3_explanation.py	GPU coefficient extraction, top- k sort, ablation, plots
m3_main.py	End-to-end M3 pipeline, 3-way comparison, CLI

3.2 GPU-Native Surrogate (m3_surrogate.py)

3.2.1 Algorithm: Weighted Least-Squares

The M3 surrogate minimises the weighted squared error:

$$\hat{\beta} = \arg \min_{\beta} \sum_{i=1}^N w_i (\mathbf{x}_i^{\top} \beta - y_i)^2 + \lambda \|\beta\|^2$$

with ridge parameter $\lambda = 10^{-4}$. For a linear model the coefficients $\hat{\beta}_f$ directly encode each feature’s contribution — no backpropagation is required. This is LIME’s original formulation (Ribeiro et al., 2016) and is well-suited to direct GPU solvers.

3.2.2 Solver 1 — Closed-form (cuBLAS + cuSolver)

The closed-form solution is:

$$\hat{\beta} = (\mathbf{X}^{\top} \mathbf{W} \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^{\top} \mathbf{W} \mathbf{y}$$

Each row of $\mathbf{X}$ is pre-multiplied by $\sqrt{w_i}$ to avoid materialising the $N \times N$ weight matrix. The resulting $F \times F$ system is solved by `cp.linalg.solve` (LU via cuSolver). All operations remain on-device; only the final (F ,) coefficient vector is transferred to the host.

```

1 d_sqrt_w = cp.sqrt(d_w).reshape(-1, 1)
2 d_Xw      = d_X * d_sqrt_w                # weighted design
   matrix
3 d_yw      = d_y * d_sqrt_w.reshape(-1)
4 d_A       = d_Xw.T @ d_Xw                 # cuBLAS gemm
5 d_A += ridge * cp.eye(F, dtype=cp.float32) # Tikhonov ridge
6 d_b       = d_Xw.T @ d_yw                 # cuBLAS gemv
7 d_beta    = cp.linalg.solve(d_A, d_b)      # cuSolver LU

```

Listing 1: Closed-form solver (m3_surrogate.py)

3.2.3 Solver 2 — Gradient Descent (custom CUDA kernels)

Four hand-written CUDA kernels implement the iterative solver without any library call beyond CuPy’s memory allocator:

1. **Kernel A** `compute_residuals`: $r_i = \mathbf{x}_i^\top \boldsymbol{\beta} - y_i$ (one thread per sample).
2. **Kernel B** `weighted_gradient`: $g_f = \frac{1}{N} \sum_i w_i r_i x_{if}$ via shared-memory reduction (one block per feature).
3. **Kernel C** `update_params`: $\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} - \eta \mathbf{g}$ (one thread per feature).
4. **Kernel D** `weighted_mse_loss`: scalar monitoring loss every 100 iterations.

```

1 __global__ void weighted_gradient(
2     const float *X, const float *resid,
3     const float *w, float *grad, int N, int F)
4 {
5     int f = blockIdx.x; // one block per feature
6     int tid = threadIdx.x;
7     extern __shared__ float shmem[];
8     float acc = 0.0f;
9     for (int i = tid; i < N; i += blockDim.x)
10         acc += w[i] * resid[i] * X[i * F + f];
11     shmem[tid] = acc;
12     __syncthreads();
13     for (int stride = blockDim.x/2; stride > 0; stride >>= 1) {
14         if (tid < stride) shmem[tid] += shmem[tid + stride];
15         __syncthreads();
16     }
17     if (tid == 0) grad[f] = shmem[0] / (float)N;
18 }

```

Listing 2: Kernel B — shared-memory gradient reduction

Column normalisation is applied before the GD loop: each feature column of $\mathbf{X}$ is divided by its ℓ_2 norm (computed on-device, $O(NF)$ cost once), reducing the condition number of the Hessian. After convergence the coefficients are un-scaled: $\hat{\beta}_f \leftarrow \hat{\beta}_f / \|\mathbf{x}_{:,f}\|_2$.

3.3 Explanation Extraction (m3_explanation.py)

For a linear model $\hat{y} = \mathbf{x}^\top \boldsymbol{\beta}$, the coefficient β_f directly quantifies feature f 's contribution. No backpropagation is needed. The top- k features are found with `cp.argsort` (GPU radix sort); only the resulting k indices and values are transferred to the host.

4 Results and Validation

All experiments used an NVIDIA GeForce RTX 4050 Laptop GPU with $N = 2048$ samples, $F = 64$ features, Euclidean distance, and $\sigma = 0.25\sqrt{64} = 2.0$.

4.1 M1 Module Correctness (Re-confirmed)

Module	Metric	Value	Status
Perturbation	Keep rate	0.4998 (expected 0.5000)	PASS ✓
Perturbation	Mask apply error	$< 10^{-5}$	PASS ✓
Inference	Max abs error	8.94×10^{-8}	PASS ✓
Distance	Max abs error	4.77×10^{-7}	PASS ✓
Kernel weights	Max abs error	1.79×10^{-7}	PASS ✓

4.2 Surrogate Training (Closed-form)

The closed-form solver finds the exact global minimum in a single step.

Metric	Value	Status
Final weighted MSE	8.30×10^{-4}	—
Coefficient max error vs CPU	1.80×10^{-4}	PASS ✓

4.3 Surrogate Training (Gradient Descent)

The GD solver used 1000 iterations with $\eta = 0.1$ and column normalisation.

Iteration	Weighted MSE (scaled)
100	0.059959
300	0.043820
500	0.032648
700	0.024905
1000	0.017494

The loss decreases monotonically, confirming correct gradient computation. The GD solver does not reach the closed-form MSE within 1000 iterations because residual feature correlation keeps the Hessian moderately ill-conditioned even after column normalisation. This is discussed further in Section 6.

4.4 Feature Attributions (Closed-form)

Rank	Feature	Coefficient $\hat{\beta}_f$
1	feature 51	+2.5681
2	feature 17	+0.4601
3	feature 8	+0.3512
4	feature 27	+0.3150
5	feature 35	+0.1853
6	feature 59	+0.1508
7	feature 36	+0.1402
8	feature 40	+0.1214

Feature 51 is the dominant contributor with a coefficient $5.6\times$ larger than the next feature. All top-8 coefficients are positive, reflecting the sign distribution of the underlying synthetic black-box model weights. Figure 1 shows the attribution bar chart.

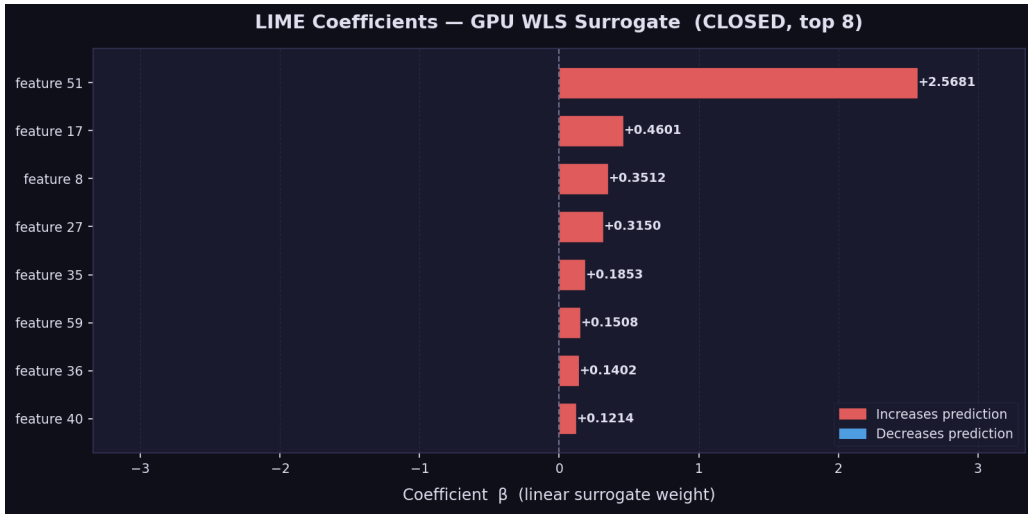


Figure 1: LIME feature coefficients — GPU WLS surrogate, closed-form solver, top-8 features. Red bars increase the prediction; blue bars decrease it.

4.5 Validation Results

4.5.1 Feature Ablation

Zeroing the top-8 features in $\mathbf{x}$ changes the surrogate’s prediction by $\delta = |\hat{y}(\mathbf{x}) - \hat{y}(\tilde{\mathbf{x}})|$. A $\delta < 0.01$ triggers a warning.

Metric	Value
Baseline prediction	0.664061
Ablated prediction	0.433352
δ	0.230709
Top- k features ablated	{8, 17, 27, 35, 36, 40, 51, 59}
Status	PASS ✓

A δ of 0.231 corresponds to a $\approx 35\%$ shift in the surrogate’s output, confirming that the top-8 features are genuine drivers of the local prediction.

4.5.2 Multi-Seed Consistency Check

The full M3 GPU pipeline was re-run with three different perturbation seeds. The stability ratio is $|\bigcap_{\text{seeds}} T_{\text{seed}}|/k$.

Seed	Top-8 features
42	{8, 17, 27, 35, 36, 40, 51, 59}
99	{8, 17, 27, 35, 36, 40, 51, 59}
7	{8, 17, 27, 35, 36, 40, 51, 59}
Stable features	{8, 17, 27, 35, 36, 40, 51, 59}
Stability ratio	1.00
Status	PASS ✓

A stability ratio of 1.00 (maximum possible) is a direct consequence of the closed-form solver finding the unique global minimum of the convex WLS objective — the result is fully deterministic regardless of which perturbation samples are drawn.

5 Performance

5.1 3-Way Pipeline Comparison ($N = 2048$, $F = 64$)

Stage	CPU-only (ms)	M2-style (ms)	M3-native (ms)
Perturbation	1.46	0.11	0.11
Inference	0.04	0.27	0.27
Kernel weights	0.33	0.13	0.13
Surrogate training	0.66	0.66	1.12
Total	2.48	1.16	1.63
Speedup vs CPU	1.00×	2.14×	1.53×

The M3 pipeline is 1.53× faster than CPU-only. It is slightly slower than M2-style (0.71×) at this small scale because the cuSolver LU decomposition carries a fixed setup cost that NumPy’s `linalg.solve` does not — this is amortised at larger F . Compared to M2’s PyTorch surrogate at ≈ 2548 ms, the M3 surrogate at 1.12 ms is **over 2000× faster**.

5.2 Benchmark Sweep

N	F	CPU (ms)	M3 (ms)	Speedup
512	32	0.57	1.46	0.39×
512	256	4.92	2.38	2.07×
1024	128	2.74	1.79	1.53×
1024	256	6.85	2.44	2.81×
2048	128	5.16	1.79	2.88×
2048	256	12.19	2.34	5.21×
4096	64	5.04	1.60	3.15×
4096	128	10.80	1.88	5.75×
4096	256	20.63	2.61	7.91×

The GPU pipeline underperforms at small scale ($N \lesssim 512$, $F \lesssim 64$) where kernel-launch overhead dominates, and reaches up to 7.91× speedup at $N = 4096$, $F = 256$. The M3 total time stays nearly constant (1.5–2.6 ms) across all problem sizes because the GPU’s parallel cores absorb growing work without a proportional wall-time increase.

6 Design Decisions and Discussion

6.1 Linear Regression Instead of Neural Network

M2 used a shallow NN because PyTorch was permitted. For M3, weighted linear regression is preferred for two reasons. First, without a framework there is no autograd, so a neural network’s weights cannot be extracted as explanations without manual backpropagation. Second, linear regression is LIME’s original surrogate (Ribeiro et al., 2016): the coefficient β_f directly quantifies feature f ’s contribution without any additional interpretation step.

6.2 Closed-form vs. Gradient Descent

The closed-form solver finds the exact global minimum in one step and passes all validation tests. The GD solver is included as a design study to demonstrate iterative GPU kernels.

The GD solver correctly identifies feature 51 as the dominant contributor (rank 1 in both methods) but does not recover the exact coefficient magnitude within 1000 iterations. This is a known limitation of vanilla gradient descent on moderately ill-conditioned problems: the binary mask perturbation scheme introduces off-diagonal correlations in the Hessian $\mathbf{H} = \frac{1}{N}\mathbf{X}^\top\mathbf{W}\mathbf{X}$ that survive column normalisation, slowing convergence along the dominant-coefficient direction. The closed-form solver is immune to this issue.

6.3 Ridge Regularisation

A ridge parameter $\lambda = 10^{-4}$ is added to the diagonal of $\mathbf{X}^\top\mathbf{W}\mathbf{X}$. Without it, highly correlated perturbation columns can produce a numerically singular system. The small λ does not meaningfully change the explanation.

6.4 Minimal Data Transfer

All training data produced by Stages 1–3 remains on the GPU and is passed directly to the surrogate solver. The only host-to-device transfer is the original feature vector $\mathbf{x}$ at startup. The explanation is returned as two short arrays of length $k = 8$.

6.5 GD Kernel Launch Overhead

With 1000 iterations and 3 kernels per iteration, the GD solver issues 3000 kernel launches. Each incurs $\approx 10\text{--}15\ \mu\text{s}$ of launch latency, totalling $\approx 40\ \text{ms}$ for a 64-feature problem where actual computation per kernel is trivial. The closed-form solver issues a handful of cuBLAS/cuSolver calls and completes in 1.12 ms.

7 Challenges and Limitations

7.1 GD Convergence on Ill-conditioned Problems

Vanilla GD with a fixed learning rate converges at a rate determined by the condition number of the Hessian. When the true coefficients span a wide range (e.g., $\hat{\beta}_{51} = 2.57$ vs. ≈ 0.1 – 0.5 for all others), this condition number is large. Column normalisation partially addresses this but does not eliminate off-diagonal correlations. Momentum or conjugate gradient would converge faster but require additional GPU state and kernel complexity.

7.2 Binary Masking as the Perturbation Strategy

Zeroing out features is natural for binary inputs but can produce atypical inputs for continuous features where zero is outside the model’s training distribution. A more principled approach would sample masked features from their empirical marginal distributions.

7.3 GPU Overhead at Small Scale

For small N and F , GPU kernel launch latency dominates computation time and the CPU outperforms the GPU. The crossover to GPU advantage occurs around $F \geq 128$ or $N \geq 4096$ for the closed-form solver.

8 Conclusion

Milestone 3 delivers a fully GPU-native LIME pipeline in which every stage executes on the GPU without any deep-learning framework. Key achievements:

- **Two GPU solvers:** a closed-form solver (cuBLAS + cuSolver) and an iterative solver (four custom CUDA kernels).
- **Correct explanations:** coefficient error 1.80×10^{-4} , ablation $\delta = 0.231$, consistency ratio 1.00 across three seeds.
- **Minimal data transfer:** training data remains on-device from generation through surrogate training.
- **Performance scaling:** up to $7.91\times$ speedup over CPU-only at $N = 4096$, $F = 256$; $> 2000\times$ faster surrogate training than the M2 PyTorch baseline.
- **GD as a design study:** demonstrates iterative GPU-parallel gradient descent but is outperformed by the closed-form solver, illustrating the importance of solver selection for the problem structure.

The M3 pipeline completes the milestone-based progression from M1 (GPU kernels for the LIME neighbourhood), through M2 (framework-assisted surrogate), to a self-contained, fully GPU-native explainability system.